

task_bridge

task_bridge

Tagline: Your task board and your source-of-truth, impeccably in sync — both ways.

Summary: task_bridge is a generic, decoupled, bidirectional task/board sync engine in Go. It keeps a project's workable-items SQLite source-of-truth in sync with its tracker docs and a remote board (first target: ClickUp; Jira/Linear planned) using deterministic last-edit-wins, dry-run-first, never-corrupt semantics.

Short description (~40 words): A project-agnostic Go submodule that bidirectionally syncs a SQLite workable-items SSoT ↔ tracker docs ↔ a remote board (ClickUp first). Deterministic last-edit-wins, dry-run-first, HMAC-verified webhooks; every credential and ID is injected by the consumer at runtime.

Long description (150-250 words): task_bridge is the connective tissue between how work is tracked internally and how it appears on a team's board. It keeps three representations of the same work in lockstep: a project's **workable-items SQLite single-source-of-truth**, its **tracker documentation**, and a **remote board** — the first supported board being ClickUp, with Jira and Linear planned as future members. Synchronization is deterministic (last-edit-wins), dry-run-first, and designed to never corrupt or lose data or leave any side out of sync. Architecturally it is a strict submodule consumed by other projects and is fully project-agnostic per the constitution's decoupling contract (§11.4.28): it ships zero project-specific values, and every credential, board/folder ID, item-key field, and DB path is injected by the consumer at runtime through pkg/config.Config. The module is cleanly layered: a CLI (reconcile/push/pull/resolve/status/conflicts/init) and a long-running daemon (webhook receiver + cron reconcile); a thin client wrapper over the MIT-licensed raksul/go-clickup; a resolver that turns board/folder URLs into IDs via live API probes (no URL-grammar guessing); a mapper between local workable items and remote task fields; a last-edit-wins sync engine with explicit conflict outcomes; and a

webhook receiver that verifies X-Signature HMAC-SHA256. It is honest about maturity: this is the P1 scaffold — layout, interfaces, entypoints, and the decoupling boundary are in place, but sync logic and live ClickUp calls are not yet implemented (every stub returns an explicit not-implemented error, per the no-fakes rule).

Why we built it: Teams keep the “real” state of work in code/docs while managers live on a board like ClickUp — and the two diverge constantly. `task_bridge` makes them one system, syncing deterministically and safely so neither side becomes stale or wrong.

Why it’s a game-changer: It treats board sync as a reusable, credential-injected library with strict data-safety guarantees (dry-run-first, last-edit-wins, HMAC-verified events) — so any project can bolt on trustworthy two-way board integration without coupling.

What’s innovative: - Three-way bidirectional sync: SQLite SSoT ↔ tracker docs ↔ remote board. - Total decoupling (§11.4.28): zero project values; all injected at runtime. - Live-API URL→ID resolution instead of fragile URL-grammar parsing. - HMAC-SHA256-verified webhook ingestion for live events.

Biggest technical challenges + how solved: - *Data safety across three sources:* solved with deterministic last-edit-wins, dry-run-first, and explicit conflict outcomes. - *Reusability without coupling:* solved via the pkg/config injection boundary (no shipped project specifics). - *Reliable board identification:* solved by resolving URLs to IDs through live API probes. - *Honest scaffolding:* solved by making unimplemented stubs return explicit not-implemented errors (no fakes).

Tech stack (why + how): - **Go** — engine, CLI (`cmd/task_bridge`), and daemon (`cmd/task_bridged`). - **SQLite** — the workable-items single-source-of-truth. - **raksul/go-clickup (MIT)** — ClickUp transport wrapper. - **HMAC-SHA256** — webhook signature verification. - **cron + webhooks** — daemon reconcile + live-event ingestion. - **pkg/config** — runtime credential/ID injection boundary.

Public links: - GitHub (vasic-digital): https://github.com/vasic-digital/task_bridge (public). - Consumed as a submodule; consumer-side plan lives in the consumer’s docs/research/clickup_integration/ (PRIVATE to that project). - Status: P1 scaffold — sync logic not yet implemented (do not present as shipped).

Suggested diagrams/illustrations (OpenDesign): 1. Three-way sync triangle: SQLite SSoT ↔ tracker docs ↔ ClickUp. 2. Decoupling boundary: consumer injects creds/IDs → generic engine. 3. Conflict resolution flow (last-edit-wins outcomes). 4. Daemon architecture: webhook receiver + cron reconcile.

Site relevance: vasic.digital (developer-tooling / integration engineering). Reusable-library angle for the engineering-depth section.

Priority tier: vasic-util-secondary

Source provenance: gh repo view vasic-digital/task_bridge README (mission, decoupling §11.4.28, layout, P1 scaffold status, HMAC webhooks, go-clickup); _analysis/github-vasic-digital.md (description, size).